

Lecture Summary

Introduction: Why Simple Trading Rules Underperform

The lecture begins by posing a critical question: why do common, simple trading rules (like moving average crossovers, RSI signals, or consecutive up/down days) often fail to be consistently profitable? These rules may appear to work initially, but they tend to break down when market conditions shift. This phenomenon is attributed to the fact that **markets are not purely random and volatility tends to cluster**, which static rules cannot adapt to.



Figure 1: SPY Daily Adjusted Close Prices (2019-2025)

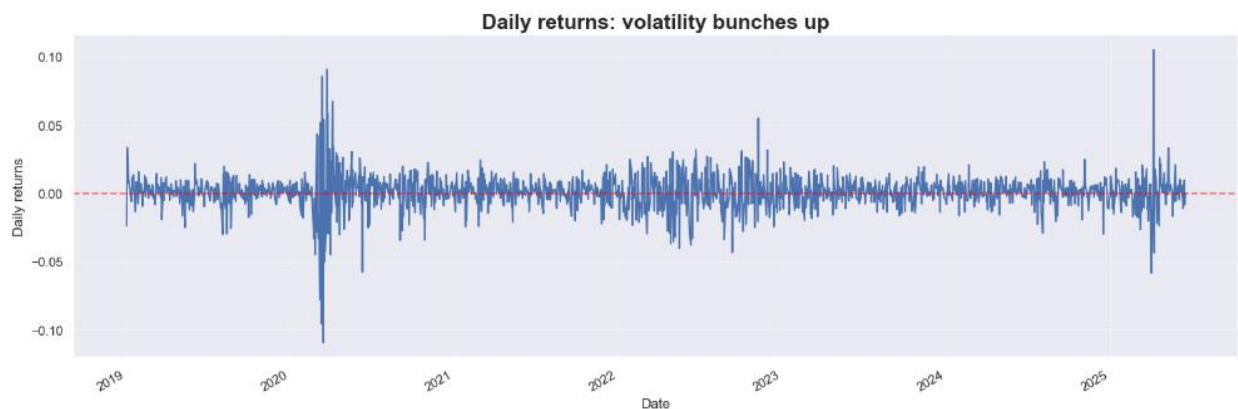


Figure 2: SPY Daily Returns (2019-2025)

The plots above illustrate the SPDR S&P 500 ETF Trust (SPY) price and its daily returns. The **volatility in returns visibly bunches up in certain periods**, suggesting that market behavior is not entirely random and that static rules will struggle.

To demonstrate this, a classic **moving average crossover strategy** (10-day short window, 30-day long window) is tested on SPY data. The strategy generates a signal (1 for long, -1 for short) based on the crossover of the short and long moving averages.

Results of Moving Average Strategy Test:

- MA strategy total returns: -5.50%
- Buy & hold total returns: 163.24%
- "Edge" (strategy - buy & hold): -168.75%

This stark comparison highlights that the **simple MA strategy significantly underperformed buy-and-hold**, reinforcing the lesson that "simple rules don't adapt to changing markets". This sets the stage for exploring statistical models that can learn patterns dynamically.

Understanding Autoregressive (AR) Models

The lecture then introduces **autoregressive (AR) models** as a more structured approach to predicting returns, shifting from intuition to a testable model. The core idea is to determine "how much of today's return can be explained using recent returns" and the reliability of that relationship.

The simplest form is the **AR(1) model**: $\text{returns_today} = \alpha + \beta \times \text{returns_yesterday} + \text{noise}$.

- **α (alpha)**: The average return when there's no momentum.
- **β (beta)**: How much yesterday's return affects today's return.
- **Noise**: The unpredictable part.

Different values of beta indicate different market behaviors:

- **$\beta > 0$** : Momentum (positive returns tend to follow positive returns).
- **$\beta < 0$** : Mean reversion (positive returns tend to be followed by negative returns).
- **$\beta = 0$** : Random walk (yesterday tells us nothing about today).

For practical application, weekly PayPal (PYPL) data from 2015 onwards is used.



Figure 3: PayPal Weekly Prices

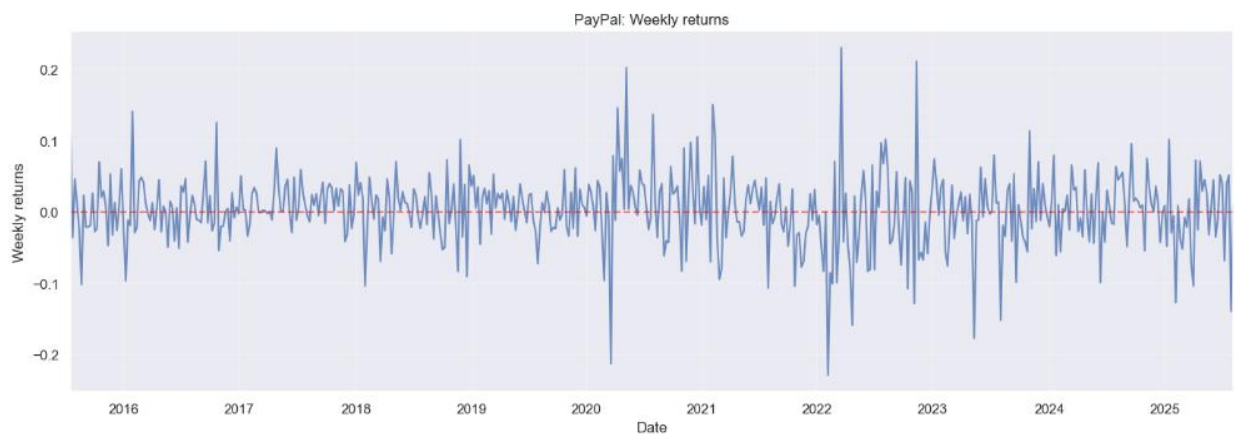


Figure 4: PayPal Weekly Returns

The data visualization for PayPal shows that **returns appear more "stable" over time than prices**, which is an early hint about stationarity. The mean weekly return for PayPal during the period is 0.0026 (13.33% annualized), with a weekly volatility of 0.0507 (36.54% annualized). This observation leads to a crucial insight: **we usually model returns, not prices**.

Stationarity: The Foundation of Time Series Models

The lecture delves into the critical concept of **stationarity**, which assumes that a time series' statistical properties (mean, variance, autocorrelation) remain constant over time. This assumption is vital because most financial time series models rely on it. Market events like the 2008 financial crisis, COVID-19, or 2022 rate hikes demonstrate that markets are dynamic, causing static models that rely on "past patterns repeating" to struggle.

To test for stationarity, two methods are employed:

1. **Rolling Statistics (Visual Check):** Examining plots of rolling mean and rolling standard deviation. For a stationary series, these lines should be relatively flat.

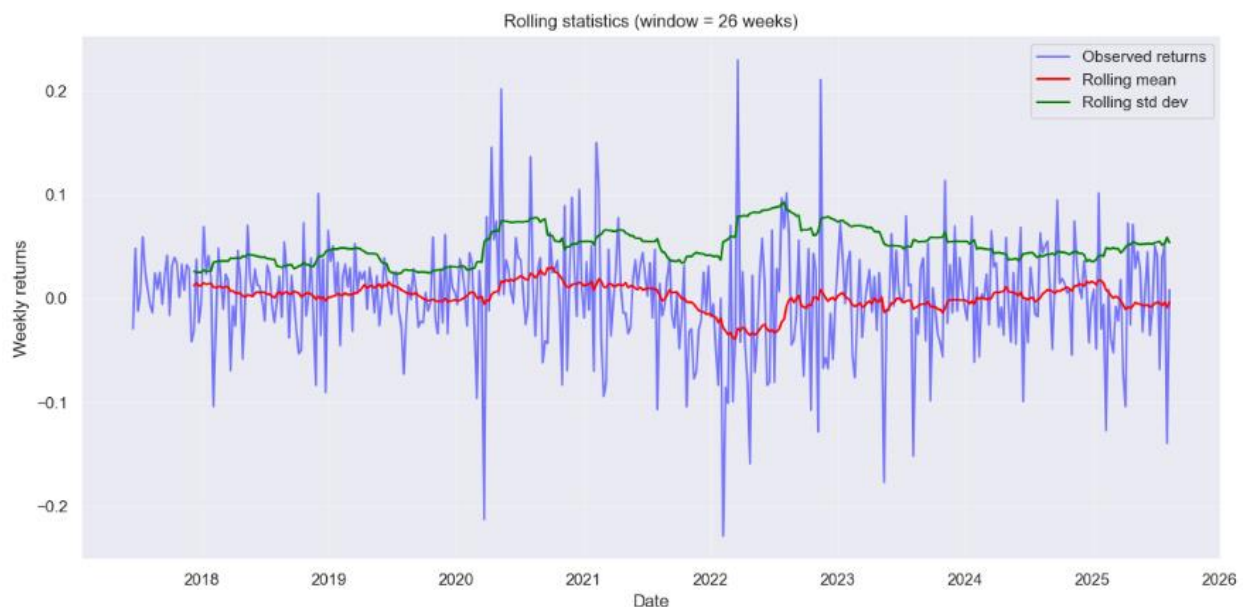


Figure 5: PayPal Weekly Returns Rolling Statistics

From the chart, the **rolling mean (red)** generally stays close to zero, indicating relative stability. The **rolling standard deviation (green)** shows some variation and spikes during volatile periods (like 2020 and 2022) but generally remains within a bounded range. These observations suggest relative stationarity, especially for the mean.

2. **Augmented Dickey-Fuller (ADF) Test (Statistical Check):** This formal test checks for a "unit root," where the null hypothesis is that the series is **not stationary**.
 - **PayPal Stock Prices:** ADF statistic: -1.8286, p-value: 0.36635. Result: **NON-STATIONARY**.
 - **PayPal Stock Returns:** ADF statistic: -24.3568, p-value: 0.00000. Result: **STATIONARY**.

These results confirm that prices are non-stationary (they trend), while returns are stationary (they fluctuate around a stable mean). This strongly supports the practice of applying AR and ARIMA models to returns or differenced prices. The ADF test on the strategy's actual returns further confirms stationarity (ADF statistic: -21.6944, p-value: 0.0000), allowing for robust modeling.

Building and Evaluating an AR(1) Trading Strategy

A trading strategy is designed using rolling AR(1) forecasts.

Strategy Design:

1. Predict next week's return using the AR(1) model.
2. Go long (buy) if the prediction is > 0 .
3. Go short (sell) if the prediction is < 0 .
4. Update predictions weekly using rolling forecasts (a 100-week training window is used).

This approach is considered more systematic, dynamic, and data-driven compared to fixed rules. The strategy building involves fitting over 400 AR(1) models with a rolling window, which takes about 26 seconds.

AR(1) Model Performance Report (PayPal Weekly Returns):

- Total returns (strategy): 18.99%
- Total returns (buy & hold): 30.50%
- Excess returns (strategy - buy & hold): -11.51%
- Sharpe ratio (strategy): 0.25
- Sharpe ratio (buy & hold): 0.28
- Win rate: 52.2%
- Maximum drawdown: -76.39%

The AR(1) strategy **underperformed buy-and-hold**.



Figure 6: AR(1) Strategy and Buy-and-Hold Equity Curves



Figure 7: AR(1) Strategy Prediction Accuracy

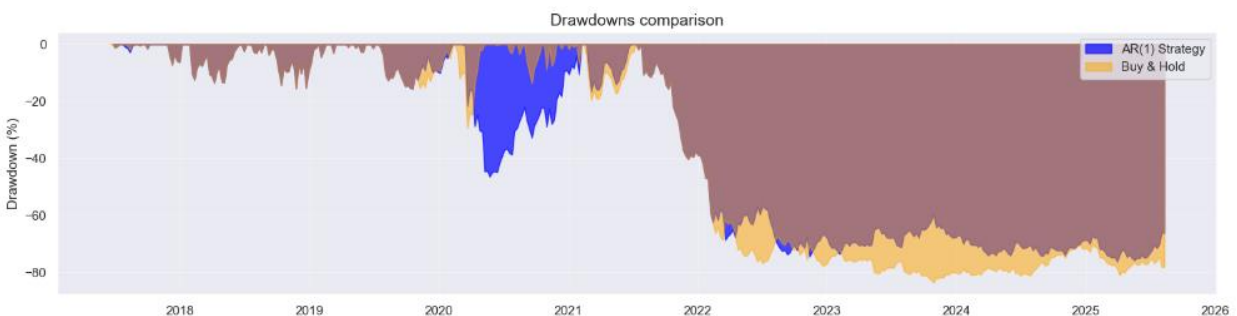


Figure 8: AR(1) Strategy and Buy-and -Hold Drawdowns

Insights from AR(1) Model Performance:

- **What went right:** Predictions weren't purely random, showing some predictive structure (average accuracy 52.6%, best 6-month accuracy 76.9%), and the model logic/execution was systematic.
- **What went wrong:** The strategy significantly **underperformed buy-and-hold**, suffered from **high drawdowns**, and was **too simplistic for complex markets**, unable to handle trends, reversals, or prediction errors. The model struggled to keep up with market movements despite weekly updates.

ARIMA Models: The Institutional Approach

To address the shortcomings of the AR(1) model, the lecture introduces **ARIMA (AutoRegressive Integrated Moving Average) models**, which investment professionals favor. ARIMA models provide:

- **AR (AutoRegressive) part:** Captures momentum and mean reversion using past returns.
- **I (Integrated) part:** Handles non-stationary data automatically by differencing (e.g., converting prices to price changes).
- **MA (Moving Average) part:** Improves predictions by incorporating past forecast errors.

This comprehensive approach allows ARIMA to capture more complex patterns, adapt to changing market trends, and refine predictions by learning from mistakes.

Choosing ARIMA Order with ACF and PACF

Before building an ARIMA model, it's necessary to select its order (p , d , q). This is done by interpreting **ACF (Autocorrelation Function)** and **PACF (Partial Autocorrelation Function)** plots. These plots help identify the number of past observations (p) and past forecast errors (q) to include, as well as the differencing order (d). For PayPal weekly returns:

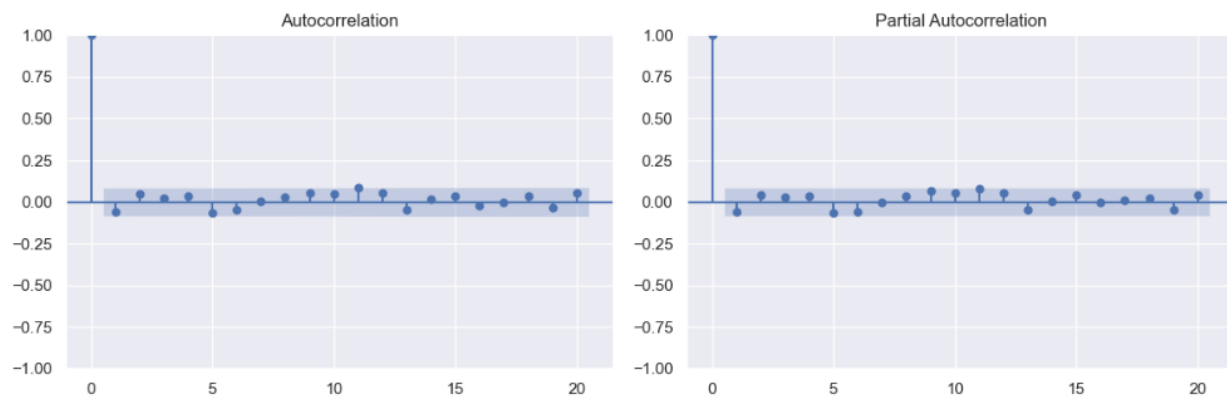


Figure 9: ACF and PACF of PayPal Weekly Returns

Based on the patterns observed in the ACF and PACF plots, **ARIMA(2,1,2)** is suggested as a good starting point for price data.

- **$p = 2$:** The model looks back **two periods** for patterns.
- **$d = 1$:** The model uses **price changes (differenced prices)**, not raw prices, to achieve stationarity.
- **$q = 2$:** The model uses the **last two forecast errors** to refine predictions.

The Case for Rolling Forecasts in ARIMA

The lecture reiterates the critical importance of **rolling forecasts**. Static models fail when market structures change, as observed during COVID-19 or rate hikes. The solution is to **continuously retrain models with "rolling windows"** using the most recent data, allowing them to adapt to evolving market conditions.

Putting It All Together: Building a Rolling ARIMA(2,1,2) Strategy

The entire modeling pipeline is then synthesized into a complete strategy:

1. Use **weekly prices**.
2. Convert to **returns**.
3. Build an **ARIMA(2,1,2) model**.
4. Choose order using **ACF and PACF**.
5. Use a **rolling window** (100 weeks) to forecast price.
6. Convert forecasts into **trading signals**.
7. Track **strategy returns**.
8. Compare with **buy-and-hold**.

A `create_arma_strategy` function is used, which fits an ARIMA model on a rolling window of historical price data and generates predictions and trading signals. This process involves fitting over 400 models and takes approximately 2 minutes. The strategy generates signals by comparing the predicted price to the last known price: `signal = 1 if forecast > train_series.iloc[-1], else -1`.

ARIMA(2,1,2) Model Performance Report (PayPal Weekly Prices):

- Total returns (strategy): 30.29%
- Total returns (buy & hold): 30.50%
- Excess returns (strategy - buy & hold): -0.21%
- Sharpe ratio (strategy): 0.28
- Sharpe ratio (buy & hold): 0.28
- Win rate: 49.3%
- Maximum drawdown: -76.01%

The ARIMA strategy's performance is notably closer to buy-and-hold than the simple AR(1) model, with similar total returns and Sharpe ratios, but it still has high drawdowns. The lecture concludes Part 1, setting the stage for Part 2, which will explore more flexible modeling frameworks to better respond to real-world market dynamics.

Home Exercise

As a final exercise, we encourage you to build and evaluate a rolling ARIMA strategy on the SPY data used earlier, compare it to the AR(1) model, and consider transaction costs. This practice helps in selecting/tuning models, implementing rolling forecast strategies, and comparing performance with realistic assumptions.